

Esame OS Internals (Cabodi)

Data: 20/06/2023

Nota Generale: Tutte le risposte SI/NO devono essere motivate. Quando i risultati sono numeri, riportare passaggi intermedi rilevanti e/o formule usate che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione a richiesta, nel quale vengono indirizzati i Byte. Il sistema ha indirizzi su 32 bit, con pagine di 4KB, e dispone di TLB (Translation Look-aside Buffer). La tabella delle pagine ("page-table") è a un solo livello. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. Un TLB miss viene gestito facendo accesso alla tabella delle pagine, gestendo l'eventuale PF, aggiornando la TLB, e ripetendo l'accesso in memoria con TLB aggiornata.

Si sa che la memoria RAM ha tempo di accesso di 100 ns, che un Page Fault ha un costo di 2.1 ms oppure 4.1 ms, a seconda del valore del modify bit relativo alla pagina scelta come vittima. Si sono valutate tre stringhe di riferimento w_1 , w_2 e w_3 , di lunghezza 100.000 (ognuna). In w_1 si osservano 10.000 TLB miss e nessun PF, in w_2 20.000 TLB miss e 10 PF (tutti con modify bit 0), in w_3 10.000 TLB miss e 20 PF, di cui 10 con modify bit 0 e 10 con modify bit 1. Si sa inoltre che le tre stringhe rappresentano il comportamento del sistema in esame, assumendo che w_2 e w_3 siano equiprobabili e con probabilità (sia w_2 che w_3) doppia rispetto a w_1 .

Si risponda alle seguenti domande:

- A) Si calcoli il numero complessivo di accessi alla TLB (compresi gli accessi ripetuti dopo un miss) eseguendo, in sequenza, le tre stringhe di riferimento. Si calcoli quindi, ESCLUDENDO I TEMPI DI PF, il tempo effettivo di accesso in RAM (*Effective Access Time* - EAT), dapprima per la sequenza delle tre stringhe e poi per il sistema che rappresentano (tenendo conto delle relative probabilità).
- A) Si considerino ora i PF. Si spieghi brevemente come va correlato il modify bit con i tempi di gestione dei PF. Si calcoli, per ognuna delle tre stringhe di riferimento e quindi per il sistema che rappresentano, il tempo effettivo di accesso in RAM, tenendo conto sia della TLB che dei PF.

Domanda 2 (File System e Allocazione Indicizzata)

Contesto: Considerare un file system basato su allocazione indicizzata nel quale si utilizza per ogni file una lista di blocchi indice. ATTENZIONE: non si tratta di organizzazione Unix-UFS basata su INODE, ma solo di lista di blocchi indice. I puntatori/indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di 4KB. Il file system risiede su una partizione del disco in cui 2TB sono riservati a blocchi di dato e blocchi di indice, a cui si aggiungono

2GB riservati a metadati (FCB, direttori, ecc., blocchi indice esclusi in quanto allocati insieme ai blocchi dato).

- A) Calcolare la massima dimensione possibile per un file (si vuole la dimensione esatta del file).
 - A) Dato un file binario di dimensione 8245 KB, calcolare quanti blocchi indice e blocchi di dato occupa il file. Calcolare anche la frammentazione interna, sia per i blocchi di dato che per quelli di indice.
 - A) Si sa che il file (vedi domanda B) contiene una sequenza di numeri reali in formato **double** (64 bit per numero). Si calcoli fino a quale blocco di indice e a quale riga (casella) di questo occorre accedere per leggere i primi 512K (dove K va inteso come 2^{10}) numeri del file. (Verificare che la dimensione sia consistente).
-

Domanda 3 (Page Table e Bit di Stato)

Contesto: Si consideri un contesto di paginazione a richiesta con sostituzione di pagine. Si risponda alle domande seguenti:

- A) Spiegare brevemente per quale/i scopo/i possono essere usati, nella Page Table, i bit denominati: *valid*, *reference*, *modify*.
- A) Si tratta di bit presenti solo nella PT o anche nella TLB? (Motivare).
- A) Si vuole realizzare una Page Table a un solo livello unica per tutti i processi. E' possibile? Se no, spiegare perché. Se sì, dire quale è il costo di una conversione pagina-frame.

2 SECTION 2: OS161

Domanda 4 (Thread e Processi in OS161)

Contesto: E' dato un sistema OS161.

- B) Si considerino le funzioni `thread_switch` e `thread_fork`. Quale viene utilizzata per creare un processo? Quale per creare un nuovo kernel thread? (Motivare).
 - B) Quando viene creato un nuovo kernel thread in OS161, questo appartiene al processo che lo ha creato o può appartenere a un diverso processo? (Motivare).
 - B) Perché la funzione `thread_exit()`, nella versione base di OS161, non può essere chiamata dopo che si è chiamata `proc_destroy` (per il processo a cui appartiene il thread)?
 - B) Cosa succede se non viene chiamata la funzione `as_destroy` al termine dell'esecuzione di un processo user? Il kernel può andare in crash? (Motivare).
-

Domanda 5 (Gestione Memoria `dumbvm` e Page Table)

Contesto: Si consideri, in OS161, la realizzazione della gestione della memoria virtuale. Si risponda alle seguenti domande:

- B) Si supponga di voler realizzare in `dumbvm` la gestione dei frame liberi mediante free-list, invece che con bitmap (come fatto nella soluzione proposta per il lab 2).
 - A1) Si dica se la free list può essere una lista di frame oppure una lista di intervalli contigui di frame liberi. (Motivare).
 - A2) Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici, logici oppure in base alla dimensione di un intervallo libero? (Motivare).
- B) Si supponga di voler invece realizzare una PAGE TABLE a livello di singolo processo. Viste le dimensioni ridotte della RAM fisica, si ipotizzi che una casella (entry) della PT occupi solo 2 Byte. Si ipotizzi di NON voler gestire un heap per il processo: il processo contiene quindi unicamente (come fatto in `dumbvm`) i segmenti di codice, di dati e lo stack. Si sa che:
 - Il segmento 1 (codice) richiede 4 pagine, inizianti all'indirizzo `0x400000`
 - Il segmento 2 (dati) richiede 3 pagine, inizianti all'indirizzo `0x500000`
 - Lo stack ha dimensione 18 pagine, che terminano, anziché al massimo indirizzo logico possibile (come fatto in `dumbvm`), all'indirizzo `0x3FFFFFFF`.
- B1) Si calcoli la dimensione complessiva dell'address space e, al suo interno, il numero di pagine effettivamente usate (valide).
- B2) Si calcoli quale sarebbe la dimensione di una page table organizzata mediante due page table standard distinte, una per i soli segmenti di codice e dato e una per lo stack.